

XML

XML es un lenguaje de marcado diseñado para almacenar y transportar datos de forma estructurada definido como estándar abierto por el W3C en 1998.

Se centra en el significado semántico y la estructura de los datos (a diferencia de HTML que se ocupa de la presentación visual.)

El término “extensible” se refiere a que permite al usuario definir sus propias etiquetas.

Casos de uso

A pesar del dominio de JSON, XML es imprescindible en varios ámbitos:

- **Sistemas empresariales:** Integraciones B2B, sistemas bancarios y ERPs.
- **Servicios web SOAP:** Utilizan exclusivamente XML para la mensajería.
- **Formatos de documentos:** Utilizado en SVG, Office Open XML (.docx, .xlsx), RSS/Atom y configuraciones diversas (Maven, Android).
- **Transporte público:** El estándar SIRI lo usa para datos en tiempo real.

Estructura y Sintaxis Básica

Un documento XML se compone de tres partes fundamentales:

- **Declaración XML:** Define la versión y la codificación del documento.
- **Elemento raíz:** Un único elemento obligatorio que contiene a todos los demás.
- **Elementos hijos:** Contienen datos dentro de etiquetas.

Reglas de Sintaxis:

- **Anidamiento:** Las etiquetas deben abrirse, cerrarse correctamente y estar bien anidadas.
- **Orden:** Se deben cerrar en el orden inverso al que se abrieron.
- **Sensibilidad a mayúsculas (Case-sensitive):** Las etiquetas de apertura y cierre deben coincidir exactamente, ya que las mayúsculas y minúsculas se consideran distintas.

Tipos de nodos del árbol

El documento se compone de varios tipos de nodos:

- **Elementos:** Bloques fundamentales que contienen datos u otros elementos.
- **Atributos:** Metadatos asociados a un elemento específico.
- **Texto:** El contenido textual dentro de los elementos.
- **Comentarios:** Notas (` `) que el procesador ignora, útiles para documentar o desactivar código temporalmente. `<!-->`
- **CDATA:** Secciones de texto literal no interpretado como marcado. Son ideales para incrustar código o incluir caracteres especiales (<, >, &) sin tener que escaparlos manualmente.

```
<![CDATA[  
    <p>Texto</p>  
]]>
```

Atributos vs. elementos

- **Reglas de los atributos:**
 - Proporcionan información adicional en la etiqueta de apertura.
 - Sus valores siempre deben ir entre comillas.
 - No se pueden repetir dos atributos iguales en un mismo elemento.
 - Deben tener un valor explícito asignado.
- **Cuándo usar cada uno:**
 - **Atributos:** Recomendados para metadatos, identificadores únicos o información corta.
 - **Elementos:** Recomendados para el contenido principal o datos que pueden repetirse o crecer.

```
<persona id="123" tipo="cliente">  
    <nombre>Juan</nombre>  
</persona>
```

Validación XML

La validación tiene como objetivo principal garantizar la integridad y consistencia de los documentos XML a través de esquemas o definiciones externas. Un documento validado asegura el cumplimiento de cuatro aspectos clave:

- **Estructura correcta:** Garantiza que el orden y la jerarquía de los elementos sean los esperados.
- **Elementos esperados:** Asegura que únicamente aparezcan los elementos definidos en el esquema.
- **Tipos de datos:** Comprueba que cada valor corresponda al tipo declarado, como números o texto.
- **Reglas de negocio:** Aplica restricciones específicas, como la definición de campos obligatorios.

- **DTD**

Un DTD (Document Type Definition) establece la estructura obligatoria de un XML, funcionando como un contrato formal entre quien produce el XML y quien lo consume.

Existen dos formas de implementarlo:

- **DTD Interno:** Se declara en el prólogo del mismo archivo XML. Es ideal para ejemplos pequeños, pero tiene la desventaja de no ser reutilizable en otros documentos.
- **DTD Externo:** Se almacena en un archivo separado con extensión .dtd y se vincula desde el XML. Es la opción recomendada para sistemas grandes porque permite su reutilización, aunque exige gestionar archivos adicionales.

CONTENIDO Y OCURRENCIAS

El DTD permite definir qué puede contener cada elemento y cuántas veces puede aparecer.

Tipos de contenido:

- **#PCDATA:** Representa datos de texto plano.
- **Elementos hijos:** Define que un elemento contiene a otros elementos en su interior.
- **EMPTY:** Indica que el elemento está vacío y no tiene contenido.
- **ANY:** Permite cualquier tipo de contenido, aunque se desaconseja su uso porque elimina el control estructural.

Indicadores de ocurrencia y orden:

- **?:** El elemento es opcional (aparece cero o una vez).
- ***:** El elemento puede aparecer cero o múltiples veces.
- **+:** El elemento es obligatorio y puede aparecer una o más veces.
- **Secuencias (,):** Los elementos deben aparecer obligatoriamente en el orden exacto en el que fueron declarados.
- **Alternativas (|):** Permite que aparezca una opción u otra, pero nunca ambas.

ATRIBUTOS

Los atributos se declaran utilizando la etiqueta `<!ATTLIST>`, donde se especifica el elemento al que pertenecen, su nombre, su tipo y su valor por defecto.

Tipos de atributos:

- **CDATA:** Permite introducir texto libre sin restricciones.
- **ID:** Define un identificador único dentro del documento.
- **IDREF:** Crea una referencia a un atributo de tipo ID existente.
- **NMTOKEN:** Exige un nombre XML válido que no contenga espacios.

Valores por defecto:

- **#REQUIRED:** Hace que el atributo sea de uso obligatorio.
- **#IMPLIED:** Convierte al atributo en opcional.

- **#FIXED "valor"**: Establece un valor fijo que no se puede modificar.

```
<!-- DTD -->
```

```
<!ELEMENT libro (titulo, autor)>
```

```
<!ATTLIST libro id ID #REQUIRED lang CDATA "es">
```

```
<!ELEMENT titulo (#PCDATA)>
```

```
<!ELEMENT autor (#PCDATA)>
```

```
<!-- XML válido -->
```

```
<libro id="L001" lang="es">
```

```
  <titulo>1984</titulo>
```

```
  <autor>George  
  Orwell</autor>
```

```
</libro>
```

ENTIDADES

Las entidades funcionan como variables o constantes de texto reutilizables que se invocan usando la sintaxis &nombre;.

- **Entidades internas:** Se definen en el propio DTD y su valor se reemplaza durante el análisis del XML.
- **Entidades externas:** Importan contenido desde un archivo externo, lo cual es útil para bloques grandes de texto. No obstante, pueden representar un riesgo de seguridad (vulnerabilidad XXE), por lo que se recomienda desactivarlas si no son estrictamente necesarias.

```
<!-- DTD -->
```

```
<!ENTITY empresa "TechCorp">
```

```
<!-- XML -->
```

```
<mensaje>Bienvenido a &empresa;</mensaje>
```

```
<!-- Resultado -->
```

```
<mensaje>Bienvenido a TechCorp</mensaje>
```

LIMITACIONES

Pese a ser el mecanismo original de validación, DTD presenta varias carencias para los estándares modernos:

- **Falta de tipos de datos:** No diferencia entre números, fechas o booleanos; todo se evalúa como texto (PCDATA).
- **Sintaxis ajena a XML:** Utiliza su propio formato de escritura, lo que lo hace incompatible con los parsers XML tradicionales.
- **Restricciones limitadas:** Es incapaz de definir patrones, rangos numéricos o reglas de cardinalidad complejas.
- **Incompatibilidad con Namespaces:** No ofrece soporte para los espacios de nombres (namespaces), los cuales son fundamentales en las arquitecturas web actuales.

Debido a estas desventajas, la industria desarrolló alternativas más robustas como XML Schema (XSD) y RELAX NG.

- **XSD**

XSD (XML Schema Definition) es un lenguaje moderno escrito en formato XML nativo que se utiliza para definir la estructura y las reglas de validación de un documento XML.

Supera ampliamente las limitaciones de DTD, garantizando la consistencia de los datos y verificando tanto la estructura como los tipos de datos adecuados, permitiendo definir los elementos, los atributos, el número de ocurrencias y añadir restricciones avanzadas a los valores.

Todo esquema XSD comienza con:

`<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">` donde xs es el prefijo del espacio de nombres que dice al procesador que esas son instrucciones de validación).

TIPOS DE DATOS PREDEFINIDOS

Una de las grandes ventajas de XSD es que evita errores asegurando que el contenido coincida con el tipo de dato esperado. Los principales tipos incluyen:

- **xs:string:** Texto libre.
- **xs:integer:** Número entero.
- **xs:decimal:** Número decimal.
- **xs:boolean:** Valores de verdadero o falso.
- **xs:date:** Formato de fecha (AAAA-MM-DD).
- **xs:time:** Formato de hora (HH:MM:SS).

Ejemplo de uso

```
<xs:element name="precio"  
    type="xs:decimal"
```

ELEMENTOS Y ATRIBUTOS

- **Elemento simple:** Es la unidad mínima de datos en un esquema y únicamente contiene texto o un valor.
- **Elemento complejo:** Permite definir estructuras jerárquicas anidadas, conteniendo otros subelementos o atributos a través de la etiqueta `<xs:complexType>`.
- **Atributos:** Se declaran dentro de los elementos complejos y su uso se define mediante los valores:
 - optional (opcional).
 - required (obligatorio).
 - prohibited (no permitido).

ESTRUCTURA Y CONTROL DE OCURRENCIAS

- **Ocurrencias:** XSD flexibiliza las repeticiones utilizando los atributos `minOccurs` y `maxOccurs` (el valor "unbounded" permite repeticiones ilimitadas).
- **sequence:** Exige que los elementos aparezcan obligatoriamente en el orden exacto definido en el esquema.

- **choice:** Modela opciones mutuamente excluyentes, permitiendo que aparezca solo uno de los elementos definidos.
- **all:** Permite que los elementos aparezcan en cualquier orden, pero exige que todos lo hagan exactamente una vez.

RESTRICCIONES CON FACETS

XSD asegura la integridad de los datos aplicando reglas y límites precisos mediante la etiqueta `<xs:restriction>`. Algunas aplicaciones comunes son:

- **Longitud mínima:** Define un límite de caracteres base mediante `<xs:minLength>`.
- **Rango numérico:** Establece topes máximos y mínimos utilizando `<xs:minInclusive>` y `<xs:maxInclusive>`.
- **Valores permitidos:** Crea listas cerradas de valores aceptados usando `<xs:enumeration>`.